



Transcript

Week 14: Model Validation and Deep Learning Fundamentals

Video 1 - Model Validation & Deep Learning Fundamentals

What good is a powerful model if it fails when it matters most? Model validation is how we separate models that just look smart from those that truly perform. And as deep learning pushes boundaries, mastering these fundamentals is what sets real-world solutions apart from academic exercises.

Key concepts in AI model development and evaluation are deeply interconnected. It begins with cross-validation, which enhances model reliability by ensuring efficient and robust use of available data. Building on this foundation, neural networks offer a structured approach to processing information, mimicking human cognition through interconnected layers. To make these networks capable of learning complex relationships, activation functions are introduced, allowing for non-linear transformations within the model. Tying all these elements together, deep learning frameworks like TensorFlow, Keras, and PyTorch provide the practical infrastructure and tools necessary to design, train, and deploy sophisticated AI models effectively.

Go ahead and explore each concept to sharpen your AI modeling skills and bring your models to life.

Video 2 - Introduction to Cross-Validation in Model Evaluation

How do you know if your model will work in the real world?

That's where cross-validation comes in. It's a powerful technique used to test how well a model performs before it ever sees real-world data. By helping us spot overfitting and assess generalization, cross-validation ensures our models are reliable, not just accurate on training data.

Cross-validation is a way to test how well a model will perform on new, unseen data. Instead of just training and testing on one split, it divides the data into multiple parts—training on some and testing on the rest—repeating this process to check how well the model generalizes. This helps spot issues like overfitting or underfitting and ensures the model is reliable enough for real-world use.

Now that we understand the value of cross-validation, let's explore some of its most widely used techniques and how they differ. One of the most common methods is K-Fold Cross-Validation, where the dataset is split into k equal parts or "folds." The model is trained and validated k times, each time using a different fold as the validation set and the remaining folds for training. This rotation provides a reliable average performance score by ensuring every part of the data is tested.

Building on that, Leave-One-Out Cross-Validation (L-O-O-C-V) takes the idea to the extreme—treating each individual data point as its own validation set. The model is trained on all other points and this process is repeated for every single point in the dataset. While this offers a very thorough assessment, it becomes computationally expensive with larger datasets.

To address the challenges of working with imbalanced data, Stratified K-Fold Cross-Validation improves upon basic K-Fold by ensuring that each fold maintains the same class distribution as the original dataset. This makes the evaluation more representative and reliable, especially when some classes are underrepresented.



Let's break down how K-Fold Cross-Validation works. Suppose we split our dataset into five equal parts, called folds. In each round, we use one fold to test the model and the remaining four folds to train it. We repeat this five times, each time using a different fold as the test set. After all five rounds, we calculate the average of the results. This method helps us measure the model's performance more accurately by testing it on different data each time.

Leave-One-Out Cross-Validation, or L-O-O-C-V, is a detailed method where each data point takes a turn as the test set. The model is trained on all remaining data and tested on just that one point. This process repeats until every point has been used for testing once. L-O-O-C-V is especially useful when you're working with small datasets, because it allows almost all of the data to be used for training in every cycle. While this technique gives a very precise view of model performance, it can be slow and demanding with larger datasets, since it runs as many training cycles as there are data points.

Let's break down L-O-O-C-V with a simple example. Imagine a dataset with five points. In each round, we pick one point to test and use the other four to train the model. First, point 1 is tested while points 2 to 5 are used for training. Then point 2 is tested, and the rest are used for training. This process repeats until every point has been tested once. Finally, we average the results from all five rounds to get the overall score. This gives us a very detailed picture of how the model performs—though it can be slow when working with large datasets, since each point requires a separate run.

Stratified K-Fold Cross-Validation is especially useful when your dataset contains imbalanced classes—where one class appears more often than another. Instead of splitting data randomly, this method keeps the class proportions the same in every fold. This ensures that each training and test set reflects the true distribution of the dataset, making performance results more accurate and fair. For example, if 80 percent of your data is class A and 20 percent is class B, each fold will maintain that 80-20 balance.

In this example, we're working with a dataset that has two classes. Class A makes up eighty percent of the data, and Class B makes up twenty percent. Stratified K-Fold Cross-Validation helps us keep this same balance in every fold. That means, when we split the data into folds, each fold still has the same ratio of Class A to Class B. We then test on one fold and train on the rest, just like regular K-Fold. This method is especially helpful when one class is much larger than the other, because it prevents the model from being biased and gives more reliable results.

There are different cross-validation methods, and choosing the right one depends on your dataset and computing power. K-Fold Cross-Validation is flexible and works well for most datasets, offering a solid performance estimate without too much computation. If you're working with a small dataset and want a detailed check, Leave-One-Out Cross-Validation—or L-O-O-C-V—is a good option, but it can be too slow for larger data. When your dataset has class imbalance, Stratified K-Fold is the way to go, because it keeps class proportions consistent in every fold. In the end, all these techniques help reduce overfitting, give a more accurate picture of your model's performance, and make the most of your data.

Let's take a quick look at how Cross-Validation works in Python using the `Sigh` kit-learn library.

In this example, we use the Iris dataset—a well-known dataset in classification tasks—and apply a Random Forest Classifier.

We then perform K-Fold Cross-Validation with five splits using the `cross_val_score` function. This allows us to calculate the score for each fold and compute the average performance.

This simple setup shows just how easy it is to include Cross-Validation in your workflow. We'll explore this further in the live session, where you'll learn how to customise and apply these techniques to real-world datasets.



Before we move on, ask yourself: which Cross-Validation method would you use for a medical dataset with rare disease cases—and how would it help avoid biased results?

Video 3 - From Machine Learning to Neural Networks

How do machines recognise faces or understand languages? Where traditional methods reach their limit, neural networks take over.

Let's explore how they emerged—and why they've changed the game.

Traditional machine learning models depend on features created by humans to make predictions.

While this works well for simpler tasks, it often struggles with more complex data, like images or language, where patterns are harder to define manually.

Neural networks overcome these limits using layered architectures that automatically learn features from raw data.

This ability to detect patterns at multiple levels enables more accurate predictions and opens the door to advanced applications like image recognition and natural language processing.

The history of neural networks stretches back to the 1940s. In 1943, McCulloch and Pitts proposed the first mathematical model of a neuron, laying the groundwork for future developments.

In 1958, Frank Rosenblatt introduced the Perceptron—a simple yet powerful model that became a key building block in early neural networks.

Progress stalled until the 1980s, when the backpropagation algorithm made it possible to train deeper, multi-layered networks efficiently.

By 2006, thanks to faster computers and large datasets, deep learning saw a major resurgence, leading to the powerful models we use today.

Let's take a closer look at the Perceptron—the simplest kind of neural network. It's made up of a single layer of neurons. Each neuron receives several inputs. These inputs are multiplied by weights, which show their importance. The neuron adds them together, and then an activation function checks if the total is high enough to trigger a response. If it is, the neuron 'fires' and gives an output of one. If not, the output is zero. This process allows the Perceptron to classify data. But keep in mind, in its basic form, it can only solve problems where the data can be clearly separated by a straight line.

The Perceptron follows a simple mathematical rule. We take each input, multiply it by its weight, then add them all together. We also add a **bias term**, which shifts the result to help adjust the model's behaviour. This total is then passed through an **activation function**—like a step function—which checks if the value is above a certain **threshold**. If it is, the output is one. If not, the output is zero. This allows the to make basic yes-or-no decisions and classify inputs into two categories.

Let's walk through how a Perceptron turns inputs into an output.

We'll use two inputs—two and three—with weights zero point five and negative zero point eight, and a bias of zero point one.

To get the weighted sum, multiply each input by its weight and add the bias:

zero point five times two, plus negative zero point eight times three, plus zero point one.

That equals negative one point three.

Now, we apply the activation function. Since we're using a step function with a threshold of zero, the output will be zero—because negative one point three doesn't cross the threshold.

This is how the Perceptron makes a decision: by checking if the weighted sum is high enough to activate the output.



Let's break down how a Perceptron separates data. Imagine a scatter plot showing two groups of points—each group is a class. The Perceptron draws a straight line between these groups. This line is the decision boundary. Everything on one side is classified as Class A, and everything on the other side as Class B. But this only works when the classes can be separated with a straight line. That's why basic Perceptrons are limited to simple, linearly separable problems. To handle more complex data, we need neural networks with more layers.

Let's look at a practical example of how a neural network works.

We'll use it to predict house prices using three features: the size of the house in square feet, the number of bedrooms, and a location rating from one to five.

To keep things simple, we'll use a single-layer network with one output.

This example will help us understand how neural networks combine inputs and weights to produce predictions—just like in real-world pricing tasks.

To set up our neural network, we start by identifying the inputs: the size of the house, the number of bedrooms, and the location rating.

Each of these features is assigned a weight— w_1 , w_2 , and w_3 —based on how much it influences the final prediction.

We also include a bias term, b , which helps adjust the output.

To calculate the predicted house price, we multiply each input by its corresponding weight, add the results together, and then add the bias.

This gives us a single predicted value, which is our model's output.

Let's walk through a step-by-step example using our neural network model to predict a house price. Suppose we have a house with a size of two thousand square feet, three bedrooms, and a location rating of four. We assign a weight of zero point five to the size, ten thousand to the number of bedrooms, and fifteen thousand to the location rating. We also include a bias term of fifty thousand.

To calculate the predicted price, we multiply each input by its assigned weight. So, zero point five times two thousand gives one thousand. Ten thousand times three gives thirty thousand. And fifteen thousand times four gives sixty thousand. When we add these three values together and include the bias, the total becomes one hundred and forty-one thousand.

So, according to the model, the predicted house price is one hundred and forty-one thousand dollars.

This diagram shows how our neural network is structured to predict house prices. Each feature—like the house's size, number of bedrooms, and location rating—is represented as an input node. These nodes are connected to a single output node, which represents the predicted price. Each connection is weighted to reflect how important that feature is in determining the price. The model calculates a weighted sum of all inputs, adds a bias term, and produces the final prediction. This setup gives us a clear view of how each feature contributes to the network's output.

Let's talk about what weights tell us in a neural network. In our house price prediction example, we see that the Location Rating has a weight of fifteen thousand, while the Size only has a weight of zero point five. This tells us that Location Rating has a much bigger influence on the predicted price than Size. During training, the model learns these weights by adjusting them to best match the data. This helps the model understand which features are more important when making predictions.

Neural networks make predictions by combining weighted inputs and a bias term. These weights reflect the importance of each input in generating the output. In our house price example, we used a simple neural network to show how this works—by assigning weights



to features like size, number of bedrooms, and location rating.

This method is highly adaptable. Neural networks are used across industries, from pricing homes to analysing complex datasets. Although we used a single-layer model for simplicity, real-world applications often involve deeper networks that can solve more complex problems.

This is just the starting point. Where could deeper networks take you—from simple predictions to solving real-world challenges with AI? Think about it!

Video 4 - Introduction to Activation Functions

What gives a neural network its spark? Activation functions are the spark behind smart decisions—without them, it's all just numbers.

Activation functions help neural networks decide when a neuron should "fire" and send a signal forward. They add non-linearity to the model, allowing it to learn more complex relationships than simple linear models can. So far, we've used the Step Function. It outputs one when the input is above a certain threshold, and zero otherwise. While it's useful for binary classification, it's limited for more complex tasks. That's why modern networks use other activation functions—we'll look at those next.

Let's take a look at other activation functions commonly used in neural networks. The **sigmoid function** transforms any input into a value between zero and one, which is helpful when predicting probabilities.

The **tan-h function**, or hyperbolic tangent, scales values between minus one and one, making it more balanced around zero.

The **ReL-U function**, short for Rectified Linear Unit, passes the input directly if it's positive and outputs zero if it's not.

Leaky ReL-U fixes the issue of inactive neurons by allowing a small output for negative values instead of zero.

Finally, the softmax function turns raw values into probabilities across multiple classes, making it ideal for classification problems with more than two outcomes.

Activation functions play a key role in building powerful neural networks. They allow the model to go beyond simple, linear relationships by introducing non-linearity. This means each layer can learn more complex features from the data. By applying activation functions at each hidden layer, the network can transform weighted inputs into meaningful outputs—enabling deeper learning and more accurate predictions.

Let's see how weighted inputs work in a simple time-based model. Here, we use the Auto-Regressive model of order two, or A-R 2. The formula is:

$Y_t \text{ equals } 1.5 \text{ plus } 0.6 \text{ times } Y_{t-1} \text{ plus } 0.3 \text{ times } Y_{t-2} \text{ plus epsilon } t.$

We're given the values of Y_{t-1} and Y_{t-2} and we assume **epsilon as 0**.

Now, substitute the values:

$Y_t \text{ equals } 1.5 \text{ plus } 0.6 \text{ times } 10 \text{ plus } 0.3 \text{ times } 8 \text{ plus } 0$

That gives us:

$Y_t \text{ equals } 1.5 \text{ plus } 6 \text{ plus } 2.4, \text{ which gives us } 9.9.$

This example shows how a model combines past values using weights to make a prediction.



Frameworks make it easier to build, train, and deploy neural networks. Instead of coding every detail from scratch, we can use tools like TensorFlow, Keras, and Pie Torch. These frameworks come with built-in components—such as layers, activation functions, loss calculators, and weight updates—that help speed up development.

They also improve performance by supporting large datasets and G-P-U acceleration. TensorFlow, developed by Google, is flexible and widely used in both industry and research. Keras runs on top of TensorFlow and is ideal for quick testing because it's simple and user-friendly. Pie Torch, often preferred in academic research, supports dynamic computation and makes debugging easier.

Let's start with something familiar. Imagine you're predicting how much you'll spend today based on your average spending over the last few days. This method smooths out daily ups and downs and gives a more stable estimate.

In time series analysis, this is similar to a Moving Average model—often called an M A model. It uses past errors, also known as shocks, to estimate future values in a sequence. This idea lays the foundation for more advanced models used in neural networks.

Let's build a simple neural network using Keras to predict a continuous value, like house prices. We define a model using the Sequential class, which lets us stack layers one after another. First, we add a hidden layer with sixteen neurons and use the ReL-U activation function. The input shape is set to three, which means we're feeding in three input features. Next, we add an output layer with one neuron and use a linear activation function, ideal for regression. To finish, we compile the model using the Adam optimiser and mean squared error as the loss function. This shows just how straightforward it is to set up a basic neural network using Keras.

Let's build a basic neural network in Pie Torch to predict a continuous value, like house prices.

We begin by importing essential Pie Torch libraries for building and training the model. Next, we define a class named **Simple Neural Network**, which inherits from Pie Torch's Module class.

Inside the constructor, we create two layers using the Linear function. The first is the hidden layer with sixteen neurons. The second is the output layer with one neuron.

In the forward method, the input goes through the hidden layer with ReL-U activation and then to the output layer.

Finally, we create the model, define mean squared error as the loss function, and use the Adam optimiser for training.

This setup gives us a clear example of how to build and train a neural network using Pie Torch.

Frameworks like Pie Torch, TensorFlow, and Keras make it much easier to build, train, and deploy neural networks.

They provide high-level application programming interfaces, or APIs, which let us create layers, apply activation functions, and optimise training with just a few lines of code. These tools support quick experimentation and offer powerful features like pre-trained models and G-P-U acceleration.

This makes complex tasks more manageable and helps us focus on designing and improving the model, rather than writing low-level code.

By using frameworks, we speed up the entire process—from training to deployment—and make neural networks more accessible to everyone.

Why just learn about neural networks when you can build one yourself? Fire up Pie Torch, TensorFlow, or Keras—and start coding your first model today!